TenaFit Testing Guide

Detailed step-by-step guide for testing the PostgreSQL, Node.js/Express, EFCT food-data ingestion, and Expo mobile app implementation.

Important: the food seed is generated from the Ethiopian Food Composition Table 2025 PDF. Do not replace it with mock food data.

1. What You Need Installed

- PostgreSQL 15 or newer, with a database user that can create extensions.

- Node.js and npm. The project was written for modern Node 18+.

- Python with pdfplumber available. In this Codex workspace, use the bundled Python path shown in commands below.

- Expo Go on your phone, or Android Studio/iOS simulator if testing locally.

- Twilio Verify credentials for real phone OTP testing.

- Google OAuth client ID for real Google sign-in testing.

2. Open the Project Folder

```
cd C:\Users\dave\Documents\Codex\2026-07-08\you-are-a-senior-full-stack
```

3. Create PostgreSQL Database

Run these in a terminal where psql is available. Change the password if you want.

```
psql -U postgres
CREATE DATABASE tenafit;
CREATE USER tenafit_app WITH PASSWORD 'tenafit_dev_password';
GRANT ALL PRIVILEGES ON DATABASE tenafit TO tenafit_app;
\q
```

If extension creation fails during migration, run the migration as a superuser or grant the needed extension permissions.

4. Configure Backend Environment

```
cd backend
copy .env.example .env
```

Open backend\.env and set these values:

Variable	Example / Meaning
DATABASE_URL	postgres://tenafit_app:tenafit_dev_password@localhost:5432/tenafit
TENAFIT_FIELD_ENCRYPTION_KEY	Use a long random secret, 64+ characters recommended.
JWT_SECRET	Use a separate long random secret.
TWILIO_*	Required only for real phone OTP.
GOOGLE_CLIENT_ID	Required only for real Google OAuth.
OPENAI_API_KEY	Optional. Meal plan generation works without it; LLM notes use it.

5. Install Backend Dependencies

```
cd backend
npm install
```

6. Run Database Migration

```
cd backend
npm run migrate
```

Expected result: Applied 001_initial_schema.sql. Then confirm tables exist:

```
psql "postgres://tenafit_app:tenafit_dev_password@localhost:5432/tenafit" -c "\dt"
```

7. Test EFCT PDF Parsing

The parser has already produced 570 food rows in this workspace. To regenerate from the provided PDF:

```
cd C:\Users\dave\Documents\Codex\2026-07-08\you-are-a-senior-full-stack
python backend\src\scripts\parse_ethiopian_fct.py "C:\Users\dave\Downloads\The_Ethiopian_Food_Composition_Table_2025_cd9308_26
```

Expected result: Parsed 570 foods and wrote backend\data\ethiopian-fct-food-data.json.

```
node -e "const rows=require('./backend/data/ethiopian-fct-food-data.json'); console.log(rows.length, rows[0])"
```

8. Seed Food Data

```
cd backend
npm run seed:food
```

Expected result: Seeded 570 EFCT foods. Confirm in PostgreSQL:

```
psql "postgres://tenafit_app:tenafit_dev_password@localhost:5432/tenafit" -c "select count(*) from food_data;"
```

9. Start Backend API

```
cd backend
npm run dev
```

In another terminal, test health:

```
curl http://localhost:4000/health
```

Expected response: {"ok":true,"service":"tenafit-api"}

10. Test Real Phone OTP Auth

Requires TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, and TWILIO_VERIFY_SERVICE_SID in backend\.env.

```
curl -X POST http://localhost:4000/auth/phone/start ^
  -H "Content-Type: application/json" ^
  -d "{\"phone\":\"+251900000000\"}"

curl -X POST http://localhost:4000/auth/phone/verify ^
  -H "Content-Type: application/json" ^
  -d "{\"phone\":\"+251900000000\",\"code\":\"123456\"}"
```

Use the real SMS code in the second command. Save the returned token as TOKEN.

11. Local Backend Testing Without OTP

If Twilio is not ready, create a local user directly in the database, then sign a JWT using your JWT_SECRET. This tests profile, check-in, and meal-plan APIs without mocking food.

```
psql "postgres://tenafit_app:tenafit_dev_password@localhost:5432/tenafit" -c "select set_config('app.field_encryption_key','YO
```

Copy the returned user id. Then generate a token:

```
cd backend
node -e "const jwt=require('jsonwebtoken'); console.log(jwt.sign({sub:'PASTE_USER_ID_HERE',authProvider:'phone'}, process.env.
```

12. Test Profile Save

```
curl -X PUT http://localhost:4000/profile ^
  -H "Content-Type: application/json" ^
  -H "Authorization: Bearer TOKEN_HERE" ^
  -d "{\"age\":29,\"sex\":\"female\",\"heightCm\":165,\"weightKg\":70,\"activityLevel\":\"moderate\",\"goalWeightKg\":63,\"med
```

Expected response includes targetCalories calculated with Mifflin-St Jeor. Sensitive values are stored encrypted in user_profiles.

13. Test Meal Plan Generation

```
curl -X POST http://localhost:4000/meal-plans/today ^
  -H "Authorization: Bearer TOKEN_HERE" ^
  -H "Content-Type: application/json"
```

Expected response: breakfast, lunch, dinner, and snack selected from the seeded food_data table only. If allergies include nuts, nut rows should be filtered out.

14. Test Weekly Check-In

```
curl -X POST http://localhost:4000/profile/weekly-check-in ^
  -H "Content-Type: application/json" ^
  -H "Authorization: Bearer TOKEN_HERE" ^
  -d "{\"currentWeightKg\":68.5,\"followedPlan\":true}"
```

Expected response includes streakCount, recalculated targetCalories, and goalReached.

15. Test Mobile App

```
cd mobile
npm install
set EXPO_PUBLIC_API_URL=http://localhost:4000
npm run start
```

- Open Expo Go and scan the QR code, or run the web target.

- Complete onboarding: age, height, weight, activity, goal, health conditions, allergies.

- Dashboard should show numbers and progress bars, not graphs.

- Use Weekly check-in and confirm the progress bar and goal-reached subscription screen.

- Meal cards should appear only when connected to an EFCT-generated backend meal plan.

16. Final Test Checklist

- Migration creates users, user_profiles, food_data, meal_plans, streaks, and subscriptions.

- food_data count is 570 after seeding from the EFCT PDF.

- Profile values are encrypted bytea fields in PostgreSQL.

- Health endpoint returns ok.

- OTP works with real Twilio credentials, or local JWT path works for backend tests.

- Meal plans use food_data rows only.

- Allergy and hypertension filters affect food retrieval.

- Weekly check-in updates streak and target calories.

- Expo screens render cleanly on mobile and use progress bars, not charts.

17. Troubleshooting

- Missing app.field_encryption_key: make sure TENAFIT_FIELD_ENCRYPTION_KEY is set in backend\.env before migration or encrypted writes.

- pgcrypto extension error: run migration as postgres superuser or grant extension permissions.

- Twilio 401/403: verify account SID, auth token, and Verify service SID.

- No meal plan: confirm food_data is seeded and select count(*) from food_data returns 570.

- Mobile cannot reach API: use your computer LAN IP instead of localhost when testing on a physical phone.